

Technology Stack

- **Mobile App:** React Native (iOS & Android native), Reanimated 3, MapLibre GL radar
- **Backend / API:** Python 3.11 (**FastAPI** Async), Node.js Gateway, WebSockets (**WSS**), REST
- **AI / ML & Graph DL:** PyTorch Geometric (**ST-GAT** Corridor Attention), ONNX Runtime (<10ms)
- **Physics Kinematics:** Newton-Davis dynamic tractive engine, grade, air drag & RK4 integrator
- **Data Streaming:** Apache Kafka (100k+ event/s KRaft), Redis 7.2 (**Pub/Sub** in-memory cache)
- **Spatial & Time-Series:** PostgreSQL 16 + **PostGIS** (1D rail chainage LRS), TimescaleDB
- **Railway Ingestion:** BEL RTIS (Navic 30s GPS), CRIS FOIS freight, COA logs, e-Caution TSR
- **Security & Hosting:** OAuth 2.0 / JWT, AES-256 GCM, Docker, Kubernetes on RailCloud / NIC

100% ZERO-HARDWARE Taps existing statutory Indian Railways GPS & CRIS feeds. No engine modifications.

Powered By



Python



PyTorch



React Native



Kafka



Redis



PostgreSQL



PostGIS



FastAPI



Docker



Kubernetes

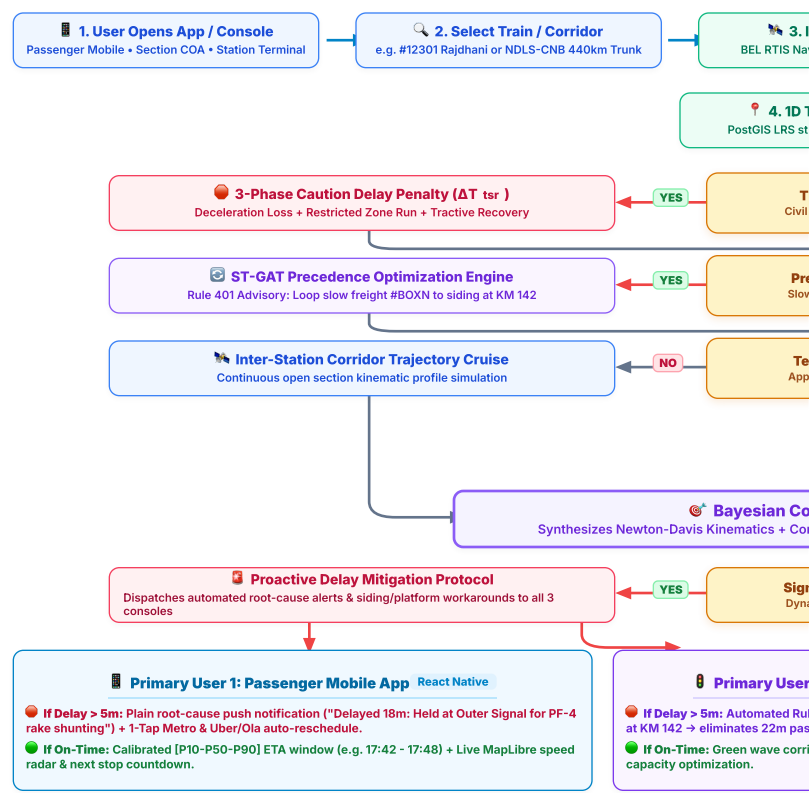


ISRO NavIC



CRIS / IR

Dynamic Operational & Multi-Stakeholder Forecasting Flowchart Closed-Loop Ground Re



1. Ingestion Stage

ZERO HARDWARE

- BEL RTIS Navic GPS**
30s live coordinates across 8,000+ locomotives
- CRIS FOIS Freight Telemetry**
Preceding freight rake tonnage, speed & section status
- COA Dispatch Records**
Section Controller line clearance, loops & crossovers
- e-Caution TSR Form T/409**
Civil Engineering temporary speed restriction slow-orders
- IMD Doppler Radar**
Real-time fog visibility <300m (GR 3.61 60km/h speed cap)

2. Processing Stage (Physi

- Newton-Davis Dynamic Accelerat**
 $a(t) = [F_{trac}(v) - R_{davis}(v)] / m$
- WAP-7 Tractive Effort & Modified**
 $F_{trac} = \min(\mu Mg, P_{ratedn}/v)$
- ST-GAT Spatial Graph Attention &**
 $a_{ij} = \text{Softmax}(\text{LeakyReLU}(a^*T))$
- Outer Signal Queuing & Platform C**
 $\Delta T_{outer} = \max(0, T_{dep}^{*pre})$